

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	HARIHARAN G	Reg. No.:	192424156
Section / Batch:	B.TECH AIDS	Date:	10.08.2026

Code of Conduct

I HARIHARAN G(192424156) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARIHARAN G

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	4	4	4	3	3	18	92
2	4	4	4	4	3	19	
3	3	4	4	4	4	19	
4	4	4	3	3	4	18	
5	4	4	3	3	4	18	

Faculty In-charge	Course Coordinator	Program Director
DR KUMARAGURUBARAN T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

```

# Training data
train_data = [
    [("I", "PRON"), ("eat", "VERB"), ("rice", "NOUN")],
    [("She", "PRON"), ("likes", "VERB"), ("music", "NOUN")],
    [("The", "DET"), ("dog", "NOUN"), ("barks", "VERB")],
    [("A", "DET"), ("cat", "NOUN"), ("sleeps", "VERB")]
]

transition_count = {}
emission_count = {}
tag_count = {}

# -----
# TRAINING
# -----

for sentence in train_data:

    previous_tag = "START"

    for word, tag in sentence:

        word = word.lower()

        if tag not in tag_count:
            tag_count[tag] = 0

        tag_count[tag] += 1

    # Emission counts

    if tag not in emission_count:
        emission_count[tag] = {}

    if word not in emission_count[tag]:
        emission_count[tag][word] = 0

    emission_count[tag][word] += 1

    # Transition counts

    if previous_tag not in transition_count:
        transition_count[previous_tag] = {}

    if tag not in transition_count[previous_tag]:
        transition_count[previous_tag][tag] = 0

    transition_count[previous_tag][tag] += 1

    previous_tag = tag

# -----
# PROBABILITY FUNCTIONS
# -----

def transition_probability(prev_tag, curr_tag):

    if prev_tag not in transition_count:
        return 0.0001

    total = sum(transition_count[prev_tag].values())

    count = transition_count[prev_tag].get(curr_tag, 0)

```

```
return count / total
```

```
def emission_probability(tag, word):
```

```
    word = word.lower()
```

```
    if tag not in emission_count:
        return 0.0001
```

```
    total = tag_count[tag]
```

```
    count = emission_count[tag].get(word, 0)
```

```
    if count == 0:
        return 0.0001
```

```
    return count / total
```

```
# -----  
# VITERBI ALGORITHM  
# -----
```

```
def hmm_pos_tag(sentence):
```

```
    words = sentence.lower().split()
```

```
    tags = list(tag_count.keys())
```

```
    viterbi = []  
    backpointer = []
```

```
    # First word
```

```
    first_scores = {}  
    first_back = {}
```

```
    for tag in tags:
```

```
        score = (  
            transition_probability("START", tag)  
            * emission_probability(tag, words[0])  
        )
```

```
        first_scores[tag] = score  
        first_back[tag] = None
```

```
    viterbi.append(first_scores)  
    backpointer.append(first_back)
```

```
    # Remaining words
```

```
    for i in range(1, len(words)):
```

```
        current_scores = {}  
        current_back = {}
```

```
        for current_tag in tags:
```

```
            best_score = -1  
            best_previous = None
```

```
for previous_tag in tags:
```

```
    score = (  
        viterbi[i - 1][previous_tag]  
        * transition_probability(previous_tag, current_tag)  
        * emission_probability(current_tag, words[i])  
    )
```

```
    if score > best_score:  
        best_score = score  
        best_previous = previous_tag
```

```
current_scores[current_tag] = best_score  
current_back[current_tag] = best_previous
```

```
viterbi.append(current_scores)  
backpointer.append(current_back)
```

```
# Find best final tag
```

```
best_tag = max(viterbi[-1], key=viterbi[-1].get)
```

```
result = [best_tag]
```

```
for i in range(len(words) - 1, 0, -1):
```

```
    best_tag = backpointer[i][best_tag]
```

```
    result.insert(0, best_tag)
```

```
return list(zip(words, result))
```

```
# -----  
# TEST  
# -----
```

```
sentence = input("Enter a sentence: ")
```

```
result = hmm_pos_tag(sentence)
```

```
print("\nPredicted tags:\n")
```

```
for word, tag in result:  
    print(word, "->", tag)
```

Pseudocode:

Sure. Based on your uploaded Hidden Markov Model (HMM) POS Tagging code, here is a moderately detailed pseudocode following the same structure.

Pseudocode – Hidden Markov Model for POS Tagging

BEGIN

1. Initialize Training Data

Store sentences containing:

- Words
- Corresponding POS tags

2. Initialize Count Tables

Create:

transition_count
emission_count
tag_count

3. TRAINING PHASE

FOR each sentence in training data:

Set previous_tag = START

FOR each word and tag in sentence:

Convert word to lowercase

Increase count of the current tag

Update emission count:

emission_count[tag][word]++

Update transition count:

transition_count[previous_tag][tag]++

Set previous_tag = current tag

4. DEFINE TRANSITION PROBABILITY

FUNCTION TransitionProbability(previous_tag, current_tag):

IF previous_tag does not exist:

RETURN small probability

Find total transitions from previous_tag

Find count of transition:

previous_tag → current_tag

RETURN count / total transitions

5. DEFINE EMISSION PROBABILITY

FUNCTION EmissionProbability(tag, word):

Convert word to lowercase

IF tag does not exist:

RETURN small probability

Find total number of words for the tag

Find count of word emitted by the tag

IF word is not found:
 RETURN small probability

RETURN count / total tag count

6. VITERBI POS TAGGING

FUNCTION HMM_POS_TAG(sentence):

 Convert sentence to lowercase
 Split sentence into words

 Get all possible POS tags

 Initialize:
 Viterbi table
 Backpointer table

 FOR the first word:

 FOR each possible tag:
 Calculate:

 Score =
 Transition Probability(START, tag)
 × Emission Probability(tag, first word)

 Store score
 Store backpointer as NULL

 FOR each remaining word:

 FOR each current tag:

 Set best_score = -1
 Set best_previous = NULL

 FOR each previous tag:

 Calculate:

 Score =
 Previous Viterbi Score
 × Transition Probability
 × Emission Probability

 IF Score > best_score:
 Update best_score
 Update best_previous tag

 Store best score
 Store best previous tag

 Select the tag having the highest
 final Viterbi score

 Backtrack using backpointers

 Assign the best tag to each word

RETURN word-tag pairs

7. TESTING PHASE

Read a sentence from the user

Call HMM_POS_TAG(sentence)

Display:

Word → Predicted POS Tag

END

Simple Flow

Training Data → Count Transitions & Emissions → Calculate Probabilities → Viterbi Algorithm → Backtracking → POS Tags

The pseudocode specifically follows the file's training, transition/emission probability, Viterbi, and testing structure.